Analyzing data on the level of cognitive processes - ESCoP 2025

Gidon T. Frischkorn, Ruhi Bhanap, and Isabel Courage

Solutions: Exercise 4 - Custom `m3` models & parameter recovery simulations

In this exercise we simulated data from a custom M3 model to then fit the same custom M3 model and evaluate parameter recovery. The idea behind parameter recovery is to evaluate how well parameters that generated data can be recovered by a model fitting routine given a certain amount of data for each subject.

```
# empty environment
rm(list = ls())

# load libraries
library("here")
library("bmm")
library("dplyr")
library("tidyr")
library("brms")
library("ggplot2")
library("tidybayes")
library("readr")
```

In this exercise we simulated data from a custom M3 model to then fit the same custom M3 model and evaluate parameter recovery. The idea behind parameter recovery is to evaluate how well parameters that generated data can be recovered by a model fitting routine given a certain amount of data for each subject.

Specify custom M3 to simulate data from

First, you need to specify the custom M3 model to simulate data from. For this, you need to a) specify the activation functions for the different response categories of your custom M3, and b) you need to set up the M3 model object that includes additional information, such as the number of response candidates in each response category, the choice rule, and the version.

We want to simulate a custom `m3` for a complex span task assuming that there is a different degree to which `item memory` and `binding memory` can be filtered/removed during the task. We assume that there are five response categories:

1. The `correct` response
2. Choosing an `other` item from the list
3. Choosing a distractor in the same position of the correct item: `distIP`
4. Choosing a distractor in other list positions: `distOP`
5. Not presented lure: `npl`

For the number of candidates in each response category, let's assume we conducted a task with set size five. That is there is one `correct` candidate and one `distIP` candidate, there are four `other` and four `distOP` candidates, and then we present additionally five `npl` options. In a real experiment this would result in participants selecting their response out of 15 retrieval candidates.

As the `choice_rule` we will use the `softmax` as it is more stable in parameter estimation.

```
# activation functions for custom model
act_funs <- bmf(
  correct ~ b + a + c,
  other ~ b + a,
  distIP ~ b + ra * a + rc * c,
  distOP ~ b + ra * a,
  npl ~ b
)

# specify the model that data should be simulated for
m3_model_custom <- m3(resp_cats = c("correct","other","distIP", "distOP", "npl"),
                      num_options = c(1,4,1,4,5),
                      choice_rule = "softmax",
                      links = list(c = "identity",
                                   a = "identity",
                                   rc = "logit",
                                   ra = "logit"),
                      version = "custom")
```

Simulate data from the custom M3

Using the activation functions you specified and the `m3` model object you generated you can simulate data for 50 subjects. For this you have to:

1. Generate subject parameters for each of the model parameters, typical parameter means are: c = 3, a = 1.5, rc = 0.3, ra = 0.8, all with an sd = 0.5
2. Pre-allocate a data frame to save the simulated data in
3. Loop over each subject and use the `rm3` function to generate data - let's start with 100 retrievals for each subject, that would mean 25 trials with testing all 5 positions in a single trial. Keep in mind that activation parameters should be positive values.

```
n_sub <- 50

# draw subject parameters: for the simple choice rule
# log link function to ensure that activation parameters are positive
sim_parms_custom <- data.frame(
  ID = 1:n_sub,
  c = rnorm(n_sub, mean = 3, sd = 0.5),
  a = pmax(0.05,rnorm(n_sub, mean = 1.5, sd = 0.5)),
  rc = brms::inv_logit_scaled(rnorm(n_sub, mean = brms::logit_scaled(0.3), sd = 0.5)),
  ra = brms::inv_logit_scaled(rnorm(n_sub, mean = brms::logit_scaled(0.8), sd = 0.5)),
  b = 0
)

# pre-allocate the data frame for simulated data
sim_data_customM3 <- data.frame(
  ID = 1:n_sub,
  correct = NaN,
  other = NaN,
  distIP = NaN,
  distOP = NaN,
  npl = NaN
)
```

```r
# merge generating parameter into sim_data data.frame
sim_data_customM3 <- merge(sim_data_customM3, sim_parms_custom, by = "ID")

# loop through all rows of the parameters that data should be simulated for
n_trials <- 500
for(i in 1:nrow(sim_data_customM3)) {

  sim_data_customM3[i,2:6] <- rm3(n = 1, size = n_trials,
                                  pars = sim_parms_custom[i,2:6],
                                  m3_model = m3_model_custom,
                                  act_funs = act_funs)
}
```

Fit the custom M3 to the simulated data

Once you have simulated data for each of the 50 subjects, we can use the simulated data and fit the same custom M3 to the simulated data. For this, we use the same activation functions, but need to add the linear models for each of the model parameters. Here, we have no experimental conditions, so it is sufficient to estimate an intercept and a random intercept over the subjects.

```r
# fit the m3 to the simulate data
m3_formula_custom <- bmf(
  c ~ 1 + (1 | ID),
  a ~ 1 + (1 | ID),
  rc ~ 1 + (1 | ID),
  ra ~ 1 + (1 | ID)
)

# fit the model with bmm
m3_fit_custom <- bmm(
  formula = act_funs + m3_formula_custom,
  model = m3_model_custom,
  data = sim_data_customM3,
  cores = 4,
  refresh = 0)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 3.3 seconds.
## Chain 2 finished in 3.3 seconds.
## Chain 3 finished in 3.3 seconds.
## Chain 4 finished in 3.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 3.3 seconds.
## Total execution time: 3.5 seconds.
```

```r
# print summary
summary(m3_fit_custom)
```

```
##   Model: m3(resp_cats = c("correct", "other", "distIP", "distOP", "npl"),
##             num_options = c(1, 4, 1, 4, 5),
##             choice_rule = "softmax",
##             version = "custom",
##             links = list(c = "identity", a = "identity", rc = "logit", ra = "logit"))
```

```
##   Links: c = identity; a = identity; rc = logit; ra = logit
## Formula: b = 0
##          correct ~ b + a + c
##          other ~ b + a
##          distIP ~ b + ra * a + rc * c
##          distOP ~ b + ra * a
##          npl ~ b
##          c ~ 1 + (1 | ID)
##          a ~ 1 + (1 | ID)
##          rc ~ 1 + (1 | ID)
##          ra ~ 1 + (1 | ID)
##    Data: (Number of observations: 50)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 50)
##                   Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(c_Intercept)       0.53      0.06     0.43     0.65 1.00     1059     2071
## sd(a_Intercept)       0.42      0.06     0.31     0.55 1.00     2247     2721
## sd(rc_Intercept)      0.52      0.09     0.36     0.70 1.00     1912     2528
## sd(ra_Intercept)      0.71      0.19     0.38     1.13 1.00     2123     2780
##
## Regression Coefficients:
##               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## c_Intercept       3.08      0.08     2.92     3.24 1.00      632     1161
## a_Intercept       1.49      0.07     1.36     1.64 1.00     2467     2804
## rc_Intercept     -1.05      0.10    -1.26    -0.86 1.00     3188     2833
## ra_Intercept      1.64      0.19     1.31     2.06 1.00     3824     2901
##
## Constant Parameters:
##                  Value
## b_Intercept       0.00
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

To assess parameter recovery of the subject parameters, we need to extract the fixed and random effects from the fitted model object. For this, you can use the `brms` functions `fixef` and `ranef`. Please note, that the random effects are deviations from the fixed intercept, so you need to add the fixed intercept to the random effects to obtain the actual subject level parameter.

```r
# extract fixed and random effects
fixFX <- brms::fixef(m3_fit_custom)
randFX <- brms::ranef(m3_fit_custom)$ID

# collect the recovered parameters
df_recPars_custom <- data.frame(
  ID = 1:n_sub,
  c = (fixFX["c_Intercept","Estimate"] + randFX[,"Estimate","c_Intercept"]),
  a = (fixFX["a_Intercept","Estimate"] + randFX[,"Estimate","a_Intercept"]),
  rc = brms::inv_logit_scaled(fixFX["rc_Intercept","Estimate"] + randFX[,"Estimate","rc_Intercept"]),
  ra = brms::inv_logit_scaled(fixFX["ra_Intercept","Estimate"] + randFX[,"Estimate","ra_Intercept"])
)
```

Quantify and Plot Parameter Recovery

Finally, we can quantify the parameter recovery with several indices. For example:

1. Calculate the correlation between the generating and recovered parameters
2. Quantify the bias in parameter estimation by calculating the mean difference between generating and recovered parameters
3. Quantify the error of approximation by calculating the Root-Mean Square Error (RMSE) -> make a quick Google search to see if you can find the formula for this.

```
cor(df_recPars_custom$c, sim_parms_custom$c)
```

```
## [1] 0.9745961
```

```
cor(df_recPars_custom$a, sim_parms_custom$a)
```

```
## [1] 0.8260804
```

```
cor(df_recPars_custom$rc, sim_parms_custom$rc)
```

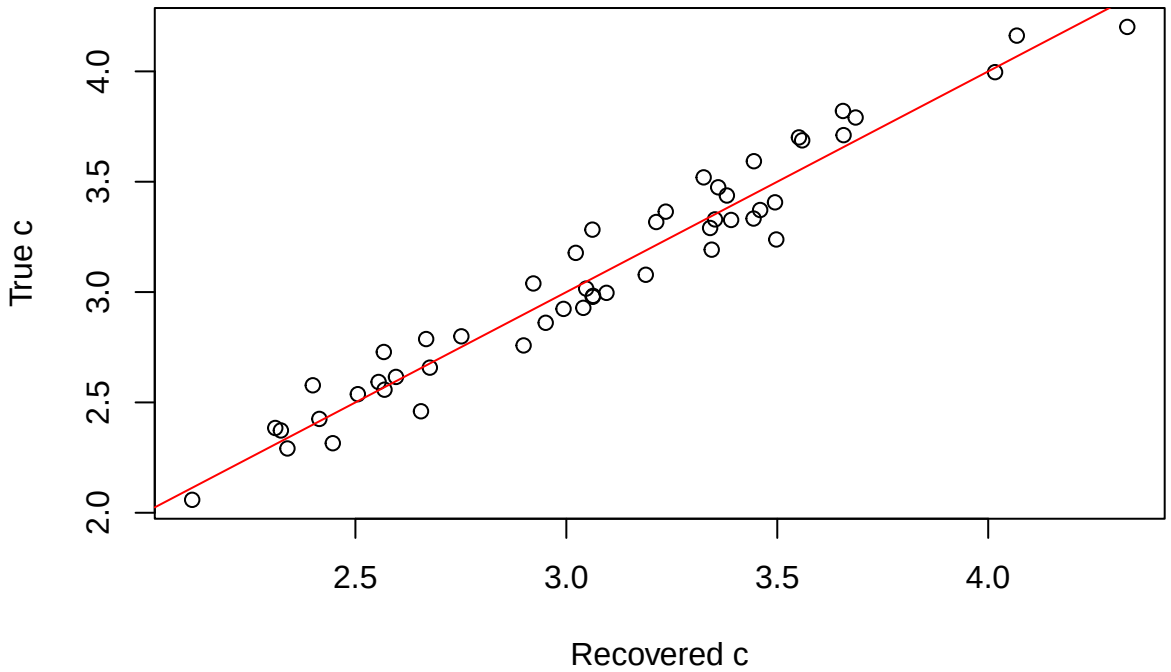
```
## [1] 0.8309194
```

```
cor(df_recPars_custom$ra, sim_parms_custom$ra)
```

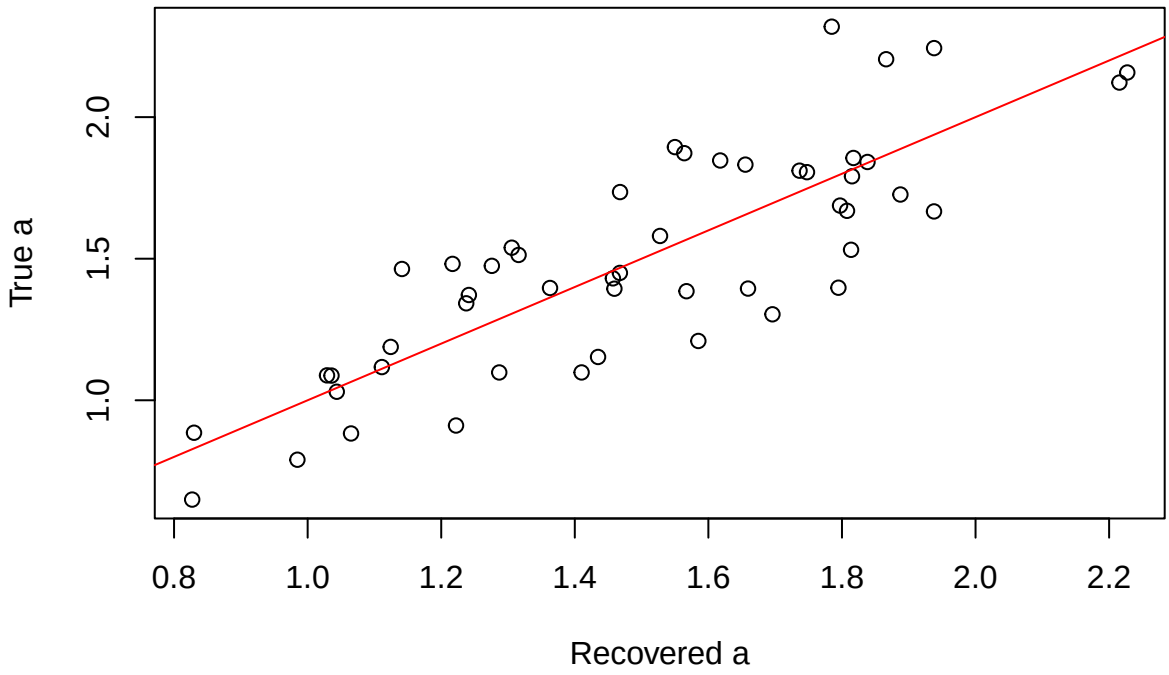
```
## [1] 0.646032
```

Then, plot the generating parameters against the recovered parameters to provide a visualization of the parameter recovery. What do you think, are 100 observations per subject enough for recovering the subject level parameters? What other aspects could influence parameter recovery?

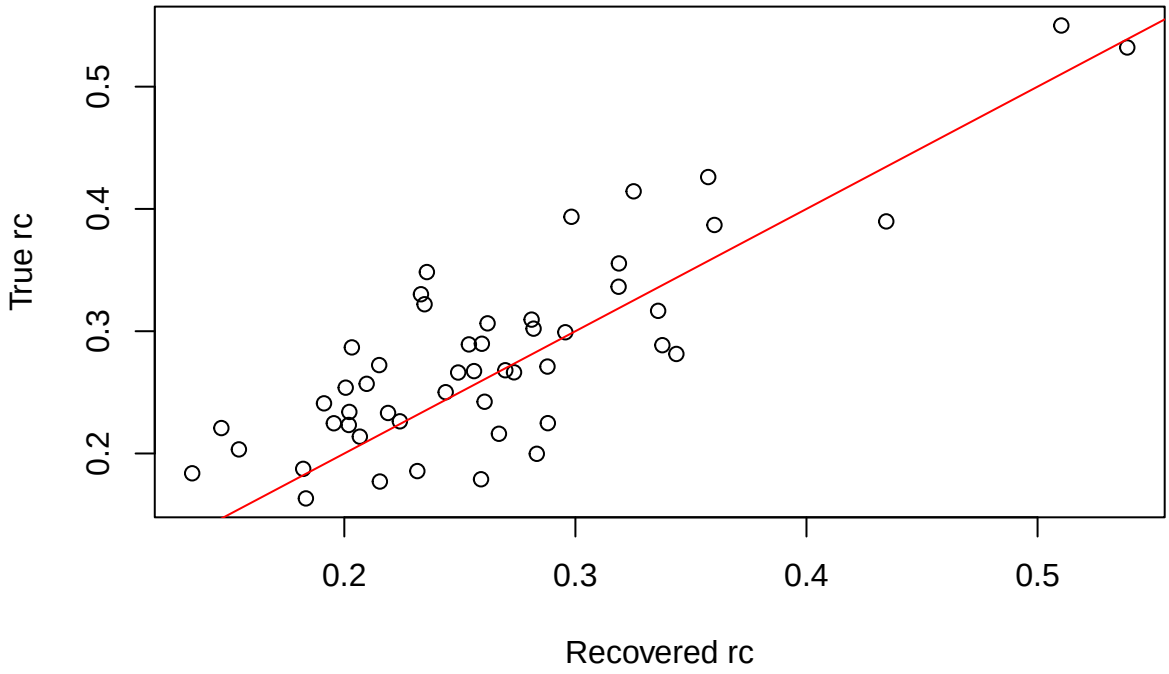
```
plot(df_recPars_custom$c, sim_parms_custom$c, xlab = "Recovered c", ylab = "True c")
abline(0,1, col = "red")
```



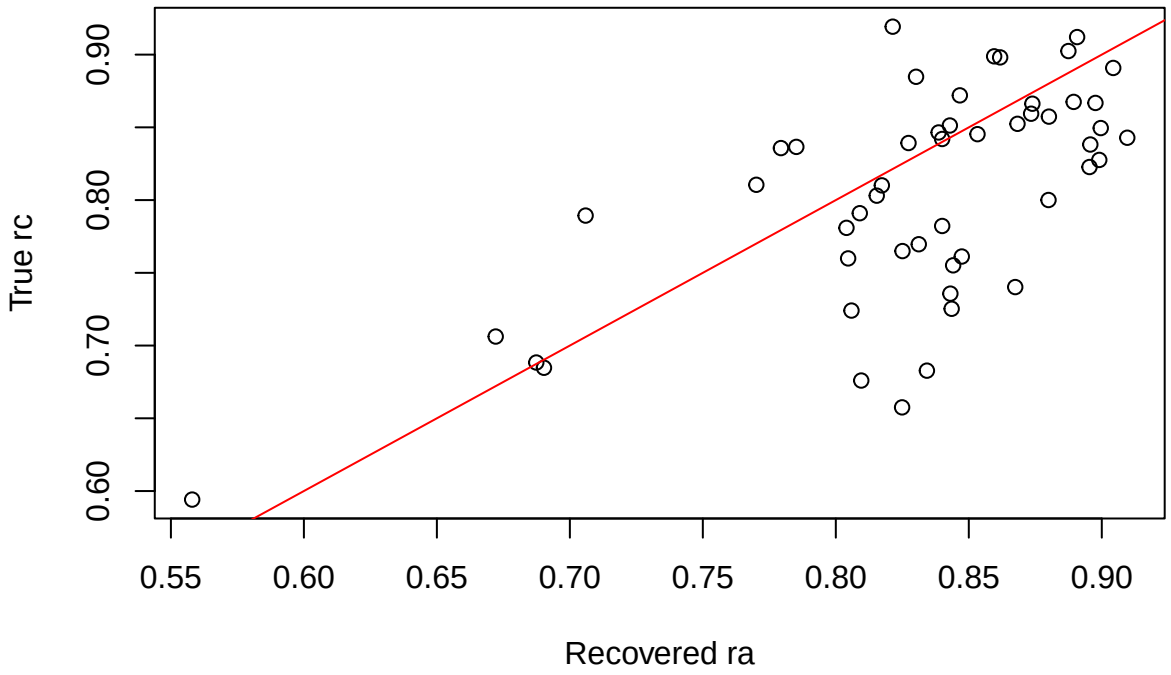

```
plot(df_recPars_custom$a, sim_parms_custom$a, xlab = "Recovered a", ylab = "True a")
abline(0,1, col = "red")
```




```
plot(df_recPars_custom$rc, sim_parms_custom$rc, xlab = "Recovered rc", ylab = "True rc")
abline(0,1, col = "red")
```




```
plot(df_recPars_custom$ra, sim_parms_custom$ra, xlab = "Recovered ra", ylab = "True rc")
abline(0,1, col = "red")
```


True rc
0.60
0.70
0.80
0.90
0.55
0.60
0.65
0.70
0.75
0.80
0.85
0.90
Recovered ra